

MODBUS GENERAL DESCRIPTION

Introduction

Modbus is a data interface protocol designed by Modicon in 1979 for use with its OEM PLC's. Due largely to its simplicity and robustness, and the fact that it was royalty free, it rapidly gained popularity and became a de facto industry standard. It is non-deterministic and of quite low integrity. As such it is generally used for non critical data transfer (e.g. vibration data, DCS interfaces, process meters etc.).

Operation

Modbus operates on a query-response cycle. A network has only one master, and a number of slaves. The maximum permitted number of slaves is 247, but this may be further limited by the transport layer (e.g. RS485 limit is 32 devices in total which equates to 1 master and 31 slaves).

The slave devices are totally passive. In isolation, they will not do anything. They require the master to issue a request to the slave. Each slave must have a unique address on the network (1 to 247). The slave will only respond if the request is valid. Invalid requests will be completely ignored by the slave device and no response whatsoever will be returned. Examples of invalid requests are: incorrect CRC, incorrect slave address (slaves only respond when a request is addressed to them).

The master is the only device that can initiate communication transactions. It does this by issuing a request (usually referred to as a "Poll" or "Telegram"). The request is normally addressed to a single specific slave. Only the addressed slave is expected to return a response. There is also a "broadcast" facility which permits the master to address all slaves. In this case the slaves will not return a response. As such the broadcast facility can only be used to write (not read) data from the slave devices.

Protocol Variants

Modbus is available in a range of different variants. Only devices using the same variant can communicate together. These variants as discussed below.

Modbus RTU (Remote Terminal Unit)

This variant communicates over a serial interface, usually RS232, RS485 or RS422. It is by far the most popular variant. Data values are encoded in 16-bit registers, in a High-Byte (X256), Low-Byte (X1) format.

For example: Take the number 12345. This would be sent as High-Byte 48 and Low-Byte 57. Note $(48 * 256) + 57 = 12345$.

Only integer values can be transferred, and are limited to the range 0 to 65535 or -32768 to 32767 for signed and unsigned numbers respectively. If decimal values need to be transferred, this can be achieved by either scaling the numbers (e.g. multiplying by 100, thus 123.45 becomes 12345 for transmission), or splitting a floating point (real) value into 2 registers and recombining it at the receiving end. Transactions are subject to strict timing constraints which restrict it to use over interfaces that cannot suffer from time

delays (e.g. direct physical connections). Data integrity is provided via a Cyclic Redundancy Check (CRC) error checksum.

Modbus ASCII

This variant communicates over a serial interface, usually RS232, RS485 or RS422. It is not a very popular variant. Data values are encoded as one byte per each ASCII character code of the value.

For example: The value 12345 would be sent as ASCII codes 49, 50, 51, 52 & 53. The data values can be best described as being a character string. As such any data value can be transferred, including decimals by including the decimal point (ASCII code 46). Transactions are not subject to strict timing constraints; as such it can be used for transmission over mediums that may be subject to random intermittent delays (e.g. modems). Data integrity is provided via a Longitudinal Redundancy Check (LRC) error checksum.

Modbus TCP

This variant communicates over an Ethernet interface, using a TCP/IP connection. This variant has become more popular over recent years, and is rapidly approaching RTU's popularity. Data is encoded in the same format as the RTU variant. In this variant, the master is referred to as a client and the slave referred to as a server. Data integrity is provided by a checksum in the TCP layer.

Data Addressing

Modbus has 4 separate data types. Each data type is accessed via fixed address ranges. These address ranges were actual memory addresses in the original Modicon PLC's. They are still used for historical reasons, but they bear no resemblance to actual memory addresses of the Siemens PLC's. Data types can hold digital (Boolean) or analogue (16-bit registers) and can allow read/write or read only access.

Data type	Address Range	Address Mapping	Access
Coils	00001 to 09998	One address per bit	Read / Write
Discrete Inputs	10001 to 10998	One address per bit	Read only
Input Registers	30001 to 39998	One address per register	Read only
Holding Registers	40001 to 49998	One address per register	Read / Write
Extended Holding Registers	40001 to 105536	One address per register	Read / Write

Most devices define their data via these addresses. However some define their data via an offset. The address can be calculated by adding the data type base address to the offset. For example an offset of 10 equates to address of 00011 for coils, 10011 for discrete inputs, 30011 for input registers and 40011 for holding registers.

Extended holding registers are a later addition to the Modbus protocol and may not be supported by all Modbus devices.

When entering the data address into the Modbus driver software, you must enter the offset value, not the address value. The offset value is calculated by subtracting the Modbus data type base address from the Modbus address.

For example: Take holding register address 40203. The offset value = $40203 - 40001 = 202$.

Function Codes

Modbus uses codes to define what data type to access and what action to perform on that data type. These are known as function codes and are sent as part of the request (poll). The original Modicon PLC's had a large number of function codes. However some were only relevant to Modicon PLC's, and modern Modbus implementations only use a selection of the most relevant and useful function codes.

Function Code	Description
01	Reads coil status values from the slave device
02	Reads discrete input status values from the slave device
03	Reads holding register data values from the slave device
04	Reads input register data values from the slave device
05	Writes a single coil status value to the slave device
06	Writes a single holding register data value to the slave device
15	Writes one or more coil status values to the slave device
16	Writes one or more holding register data values to the slave device

Exception Error Responses

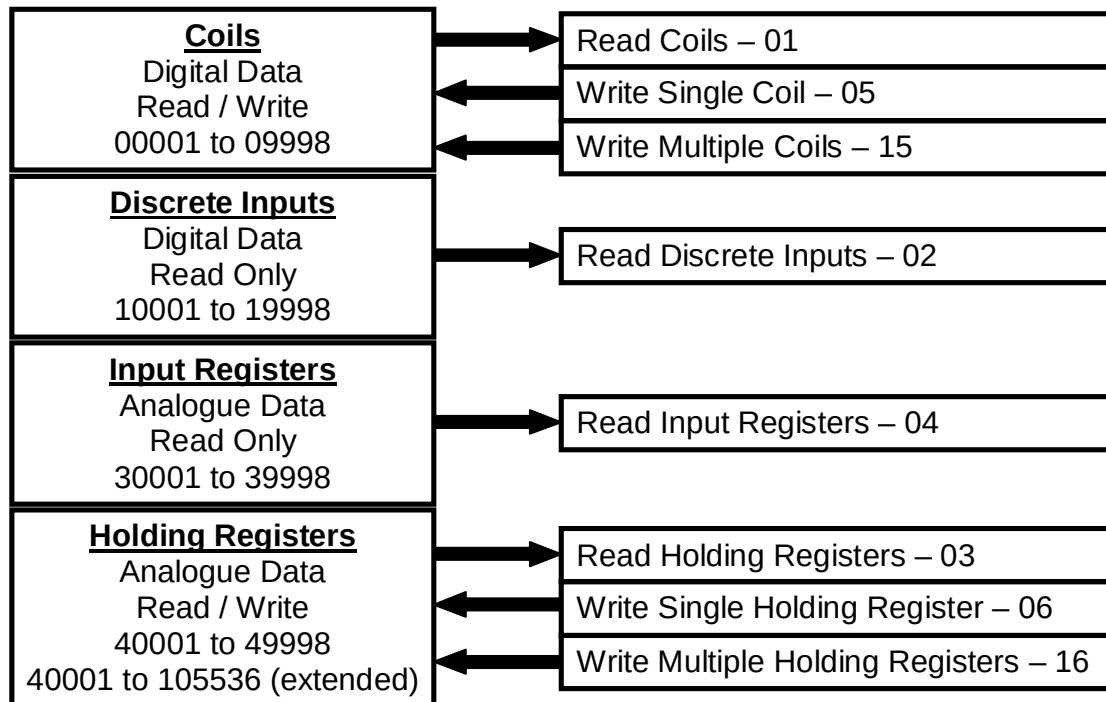
These are responses returned by the slave to the master and are sent when the slave receives a valid request, but the request cannot be handled for some reason. An exception response is identified by the returned function code to the master being a modified value of the issued function code (normal responses return identical function codes). The returned function code is defined as the normal function with 80 hexadecimal added to it. The exception responses also carry an error code to indicate the problem with the master's request. These error codes and suggested remedial actions are shown in the table below.

Exception Error Code	Description	Remedial Action
1	The function code in the request is not supported by the slave.	Reconfigure the master to use a supported function code.
2	The Modbus offset value (start address) lies outside the limits of the slaves address space or start address plus number of data points extends beyond the limits of the slaves address space.	Modify the start address offset and / or number of requested data points in the master's request.
3	The data value in the request is not permitted in the slave (e.g. single coil force value not 255 or 0). Most likely caused by a packet corruption or parameterization error in the master.	Check the master's parameterization and correct as necessary. Check that good quality cable has been used and that connections are secure and correctly terminated.
4	The slave has suffered a catastrophic error which not recoverable (e.g. hardware failure).	Service the slave device.
5	The slave has accepted the command and is processing it but a long duration of time is	None. The Siemens Modbus driver does not allow the use

	required to do so. This response is specific to remote programming commands of original Modicon PLC's and therefore should not been. seen during normal operation	of the function code for issuing of remote programming commands. function
6	The slave is engaged in a long-duration program command. This response is specific to remote programming commands of original Modicon PLC's and therefore should not been seen during normal operation.	None. The Siemens Modbus driver does not allow the use of the function code for issuing of remote programming commands.
7	The slave cannot perform the program function received in the query (specifically function codes 13 & 14). The Siemens Modbus driver does not support these function codes so this response should not be seen in normal operation.	None. The Siemens Modbus driver does not support these function codes.
8	The slave attempted to read extended memory, but detected a parity error in memory. This response is specific to Modicon PLC's and as such should not be seen during normal operation.	None. The Siemens Modbus driver does not support the function codes that can cause this error.
10	The gateway is unable to allocate an internal communication path from the input port to the output port for processing the request. This usually means that the gateway is configured incorrectly or is overloaded. Returned on TCP communications only.	Try repeating the request. If the error is permanent, then check that the TCP connection has been correctly established.
11	Indicates that the target device has failed to respond to a request from the gateway. Usually means that the device is not present on the network or has failed for some reason. Returned on TCP communications only.	Try repeating the request. If the error is permanent, then check that the TCP connection has been correctly established and that the slave device is operational.

Function Code / Modbus Address Quick Reference

Shown below is a diagram that acts as a quick reference to the Modbus function codes and addresses.



Communication Processors

Siemens make a range of communication processor (CP) cards. They are divided into serial and Ethernet types. To use Modbus on the serial CP cards, you must install the loadable Modbus drivers. These drivers require a security dongle to function. Only suitable CP cards can be loaded with the Modbus drivers; these being CP341 and CP441-2. Separate drivers are required for [Modbus RTU master](#), [Modbus RTU slave](#), [Modbus ASCII master / slave](#)

To use Modbus TCP on the Ethernet CP cards you will need the Open Modbus TCP Software. No security dongle is required to use this driver. However, a security code is needed to activate the software.

Ordering Details

[6ES7870-1AA01-0YA0](#) - dongle (Master) plus CD software

[6ES7870-1AA01-0YA1](#) - dongle (Master)

[6ES7870-1AB01-0YA0](#) - dongle (slave) plus CD software

[6ES7870-1AB01-0YA1](#) - dongle (slave)

[Open Modbus TCP](#)